

TRẦN ĐÌNH QUẾ

GIÁO TRÌNH

**KIẾN TRÚC
VÀ THIẾT KẾ PHẦN MỀM**

HÀ NỘI - 2017

MỤC LỤC

LỜI GIỚI THIỆU	i
CHƯƠNG 1: KHÁI NIỆM CƠ BẢN VỀ KIẾN TRÚC PHẦN MỀM	1
1.1 ĐỘ PHỨC TẠP PHẦN MỀM	1
1.1.1 Độ phức tạp phần mềm là gì?	1
1.1.2 Độ phức tạp và thành phần phần mềm	1
1.1.3 Ảnh hưởng của độ phức tạp phần mềm	2
1.2 KIẾN TRÚC PHẦN MỀM	3
1.3 MỘT SỐ KHÁI NIỆM CƠ BẢN	4
1.3.1 Kiến trúc	5
1.3.2 Thành phần	5
1.3.3 Kết nối	6
1.3.4 Cấu hình	7
1.3.5 Kiểu kiến trúc	7
1.3.6 Mẫu kiến trúc	7
1.3.7 Mô hình kiến trúc	10
1.4 KẾT LUẬN	10
BÀI TẬP	10
CHƯƠNG 2: THIẾT KẾ KIẾN TRÚC VÀ CÁC KIỂU KIẾN TRÚC	12
2.1 TIẾN TRÌNH THIẾT KẾ KIẾN TRÚC	12
2.1.1 Hiểu được vấn đề cần giải quyết	13
2.1.2 Xác định các phân tử thiết kế và quan hệ của chúng	13
2.1.3 Đánh giá thiết kế kiến trúc	13
2.1.4 Biến đổi thiết kế	14
2.2 CÁC NGUYÊN LÝ THIẾT KẾ KIẾN TRÚC	14
2.2.1 Kiến trúc phần mềm tổng quát	14
2.2.2 Nguyên lý thiết kế cơ bản	15
2.3 CÁC KIỂU KIẾN TRÚC	16
2.3.1 Kiểu kiến trúc Client/Server	16
2.3.2 Kiểu kiến trúc dựa vào thành phần	18
2.3.3 Kiểu kiến trúc hướng miền	19
2.3.4 Kiểu kiến trúc phân tầng	20
2.3.5 Kiểu kiến trúc bus thông điệp	22

2.3.6 Kiểu kiến trúc N-tầng.....	23
2.3.7 Kiểu kiến trúc hướng đối tượng	24
2.3.8 Kiểu kiến trúc hướng dịch vụ.....	26
2.4 KẾT LUẬN.....	27
BÀI TẬP	27
CHƯƠNG 3: MÔ HÌNH HÓA KIẾN TRÚC.....	28
3.1 KHÁI NIỆM VỀ MÔ HÌNH KIẾN TRÚC	28
3.1.1 Yêu cầu của mô hình hóa kiến trúc	28
3.1.2 Kiểu kiến trúc và mô hình hóa	29
3.1.3 Một số đặc trưng của mô hình hóa kiến trúc.....	30
3.1.4 Tính phức tạp của mô hình hóa	31
3.1.5 Tính chất của các khung nhìn.....	33
3.2 KỸ THUẬT MÔ HÌNH.....	34
3.2.1 Một số kỹ thuật mô hình	34
3.2.2 Đánh giá các kỹ thuật mô hình.....	35
3.3 MÔ HÌNH HÓA KIẾN TRÚC VỚI UML	35
3.3.1 Ngôn ngữ mô hình thống nhất UML.....	35
3.3.2 Biểu diễn kiến trúc với UML	36
3.3.3 Kiến trúc logic.....	38
3.3.4 Kiến trúc vật lý.....	41
3.4 KẾT LUẬN.....	42
BÀI TẬP	42
CHƯƠNG 4: KIẾN TRÚC PHẦN MỀM DỰA TRÊN THÀNH PHẦN.....	43
4.1 GIỚI THIỆU	43
4.1.1 Từ lập trình hướng đối tượng đến lập trình hướng thành phần	43
4.1.2 Khái niệm thành phần	45
4.2 PHÁT TRIỂN PHẦN MỀM DỰA TRÊN THÀNH PHẦN	46
4.2.1 Hướng dẫn thiết kế thành phần	46
4.2.2 Phân bố thành phần theo tầng.....	46
4.3 CƠ SỞ HẠ TẦNG CỦA PHÁT TRIỂN PHẦN MỀM HƯỚNG THÀNH PHẦN	49
4.4 BIỂU DIỄN KIẾN TRÚC VÀ THÀNH PHẦN VỚI UML	50
4.4.1 Mô hình thành phần.....	50
4.4.2 Biểu đồ triển khai	51
4.5 KẾT LUẬN.....	52
BÀI TẬP	53
CHƯƠNG 5: MÔ HÌNH THÀNH PHẦN VỚI EJB	54

5.1. KIẾN TRÚC EJB.....	54
5.1.1. Tổng quan về kiến trúc EJB và J2EE platform	54
5.1.2. J2EE Server	54
5.1.3. Container	55
5.1.4. Thành phần EJB	56
5.2. MÔ HÌNH THÀNH PHẦN CỦA EJB.....	56
5.2.1. Tổng quan về EJB 2.x	56
5.2.2. EJB 3.0	57
5.2.3 Bean phiên.....	58
5.2.4. Thành phần dịch vụ EJB	59
5.2.5 Bean thực thể.....	60
5.2.6. Bean hướng thông điệp MDB (Message-Driven Beans).....	68
5.3 MÔ HÌNH KẾT NỐI CỦA EJB.....	69
5.3.1 Kết hợp các kết nối đồng bộ.....	70
5.3.2 Kết nối lỏng lẻo không đồng bộ	70
5.3.3 Truyền thông từ xa và cục bộ.....	70
5.3.4 Các tham chiếu đối tượng đến các bean thực thể.....	70
5.3.5 Kết hợp các kết nối giữa các bean thực thể.....	70
5.4. MÔ HÌNH TRIỂN KHAI EJB.....	72
5.5 CASE STUDY	74
5.5.1. Tạo giỏ hàng.....	74
5.5.2. Xây dựng Converter với Netbean.....	78
5.6. KẾT LUẬN.....	82
BÀI TẬP	82
CHƯƠNG 6: MÔ HÌNH THÀNH PHẦN .NET	84
6.1 GIỚI THIỆU	84
6.1.1 Tổng quan về .NET framework.....	84
6.1.2. Cơ sở của .NET framework – CLR.....	85
6.1.3. Thư viện lớp của .NET	87
6.2. MÔ HÌNH THÀNH PHẦN CỦA .NET	88
6.3. MÔ HÌNH KẾT NỐI.....	93
6.3.1. Thành phần kết nối .NET	93
6.3.2. Kết nối các thành phần bằng Sự kiện (Event) và Ủy nhiệm (Delegate).....	94
6.3.3. Các bộ kết nối từ xa cho các thành phần phân tán .NET.....	96
6.3.4. Gọi không đồng bộ từ xa giữa các thành phần phân tán .NET.....	99

6.4 MÔ HÌNH TRIỂN KHAI THÀNH PHẦN .NET	100
6.4.1. Triển khai riêng	101
6.4.2 Triển khai chia sẻ chung	101
6.5 KẾT LUẬN	104
BÀI TẬP	104
CHƯƠNG 7 : KIẾN TRÚC VÀ MẪU THIẾT KẾ	105
7.1 KHÁI NIỆM MẪU THIẾT KẾ	105
7.2 ĐỊNH DẠNG MẪU THIẾT KẾ	106
7.3 PHÂN LOẠI MẪU THIẾT KẾ	107
7.4 SỬ DỤNG MẪU THIẾT KẾ	109
7.4.1 Khi nào sử dụng mẫu thiết kế?	109
7.4.2 Sử dụng mẫu thiết kế như thế nào?	109
7.5 KẾT LUẬN	110
BÀI TẬP	111
CHƯƠNG 8: CÁC MẪU THIẾT KẾ TẠO DỰNG	112
8.1 MẪU THIẾT KẾ FACTORY METHOD	112
8.1.1 Đặt vấn đề	112
8.1.2 Cấu trúc mẫu	112
8.1.3 Tình huống áp dụng	113
8.1.4 Ví dụ	114
8.2 MẪU THIẾT KẾ SINGLETON	116
8.2.1 Đặt vấn đề	116
8.2.2 Cấu trúc mẫu	116
8.2.3 Tình huống áp dụng	116
8.2.4 Ví dụ	116
8.3 MẪU THIẾT KẾ ABSTRACT FACTORY	119
8.3.1 Đặt vấn đề	119
8.3.2 Cấu trúc mẫu	119
8.3.3 Tình huống áp dụng	120
8.3.4 Ví dụ	120
8.4 MẪU THIẾT KẾ BUILDER	121
8.4.1 Đặt vấn đề	125
8.4.2 Cấu trúc mẫu	125
8.4.3 Tình huống áp dụng	126
8.4.4 Ví dụ	126

8.5 MẪU THIẾT KẾ PROTOTYPE.....	128
8.5.1 Đặt vấn đề.....	128
8.5.2 Cấu trúc mẫu	128
8.5.3 Tình huống áp dụng.....	128
8.5.4 Ví dụ.....	129
8.6 KẾT LUẬN.....	129
BÀI TẬP	129
CHƯƠNG 9: CÁC MẪU THIẾT KẾ CẤU TRÚC	130
9.1 MẪU ADAPTER.....	130
9.1.1 Đặt vấn đề.....	130
9.1.2 Cấu trúc mẫu	130
9.1.3 Tình huống áp dụng.....	131
9.1.4 Ví dụ.....	131
9.2 MẪU BRIDGE	131
9.2.1 Đặt vấn đề.....	131
9.2.2 Cấu trúc mẫu	132
9.2.3 Tình huống áp dụng.....	132
9.2.4 Ví dụ.....	133
9.3 MẪU COMPOSITE	135
9.3.1 Đặt vấn đề.....	135
9.3.2 Cấu trúc mẫu	135
9.3.3 Tình huống áp dụng.....	136
9.3.4 Ví dụ.....	136
9.4 MẪU DECORATOR.....	137
9.4.1 Đặt vấn đề.....	137
9.4.2 Cấu trúc mẫu	138
9.4.3 Tình huống áp dụng.....	138
9.5 MẪU FAÇADE	141
9.5.1 Đặt vấn đề.....	141
9.5.2 Cấu trúc mẫu	142
9.5.3 Tình huống áp dụng.....	142
9.5.4 Ví dụ.....	143
9.6 MẪU FLYWEIGHT	145
9.6.1 Đặt vấn đề.....	145
9.6.3 Tình huống áp dụng.....	146

9.6.4 Ví dụ.....	146
9.7 MẪU PROXY.....	148
9.7.1 Đặt vấn đề.....	148
9.7.2 Cấu trúc mẫu	148
9.7.3 Tình huống áp dụng.....	149
9.7.4 Ví dụ.....	149
9.8 KẾT LUẬN.....	151
BÀI TẬP	151
CHƯƠNG 10: CÁC MẪU THIẾT KẾ HÀNH VI.....	153
10.1 MẪU CHUỖI TRÁCH NHIỆM.....	154
10.1.1 Đặt vấn đề.....	154
10.1.2 Cấu trúc mẫu	154
10.1.3 Tình huống áp dụng.....	155
10.1.4 Ví dụ.....	155
10.2 MẪU COMMAND.....	158
10.2.1 Đặt vấn đề.....	158
10.2.2 Cấu trúc mẫu	159
10.2.3 Tình huống áp dụng.....	160
10.2.4 Ví dụ.....	160
10.3 MẪU ITERATOR.....	161
10.3.1 Đặt vấn đề.....	161
10.3.2 Cấu trúc mẫu	161
10.3.3 Tình huống áp dụng.....	162
10.3.4 Ví dụ.....	162
10.4 MẪU INTERPRETER.....	163
10.4.1 Đặt vấn đề.....	163
10.4.2 Cấu trúc mẫu	164
10.4.3 Tình huống áp dụng.....	164
10.4.4 Ví dụ.....	165
10.5 MẪU MEDIATOR	167
10.5.1 Đặt vấn đề.....	167
10.5.2 Cấu trúc mẫu	168
10.5.3 Tình huống áp dụng.....	168
10.6 MẪU MEMENTO	169
10.6.1 Đặt vấn đề.....	169

10.6.2 Cấu trúc mẫu	169
10.6.3 Tình huống áp dụng.....	169
10.7 MẪU OBSERVER.....	170
10.7.1 Đặt vấn đề.....	170
10.7.2 Cấu trúc mẫu	170
10.7.3 Tình huống áp dụng.....	171
10.8 MẪU STATE.....	171
10.8.1 Đặt vấn đề.....	171
10.8.2 Cấu trúc mẫu	171
10.8.3 Tình huống áp dụng.....	171
10.9 MẪU STRATEGY	172
10.9.1 Đặt vấn đề.....	172
10.9.2 Cấu trúc mẫu	172
10.9.3 Tình huống áp dụng.....	172
10.10 MẪU TEMPLATE METHOD	172
10.10.1 Đặt vấn đề.....	172
10.10.3 Tình huống áp dụng.....	173
10.11 MẪU VISITOR	173
10.11.1 Đặt vấn đề.....	173
10.11.2 Cấu trúc mẫu	174
10.11.3 Tình huống áp dụng.....	175
10.12 KẾT LUẬN.....	176
BÀI TẬP	176
CHƯƠNG 11: CASE STUDY	177
11.1 CASE STUDY 1: THIẾT KẾ CƠ CHẾ TRUY NHẬP DỮ LIỆU	177
11.1.1 Cấu trúc của DAO	177
11.1.3 Hệ quản lý dữ liệu khách hàng	179
11.2 CASE STUDY 2: HỆ QUẢN LÝ BÁN SÁCH TRỰC TUYẾN BOOKSTORE	184
TÀI LIỆU THAM KHẢO	198

LỜI GIỚI THIỆU

Kiến trúc phần mềm được xem là trung tâm của nghiên cứu và phát triển các hệ thống phần mềm cỡ lớn hiện nay. Quá trình của phát triển được xoay quanh khái niệm kiến trúc nhằm mục đích phát triển sản phẩm một cách có hiệu quả, năng suất và chất lượng cao. Tài liệu này được dành cho sinh viên năm cuối khi học môn kiến trúc và thiết kế phần mềm. Tài liệu được xây dựng dựa trên cơ sở là bạn đọc đã có được kiến thức và kỹ năng cơ bản về Phân tích và thiết kế phần mềm, lập trình hướng đối tượng và có thể biểu diễn cấu trúc và hành vi của hệ thống với ngôn ngữ mô hình UML.

Mục đích của tài liệu là nâng cao kiến thức và kỹ năng về thiết kế phần mềm cho sinh viên ngành công nghệ thông tin. Nội dung tập trung vào các cách tiếp cận liên quan đến kiến trúc và thiết kế phần mềm cũng như mẫu thiết kế. Phần kiến trúc phần mềm chủ yếu trình bày cách tiếp cận dựa trên thành phần và các công nghệ thành phần được sử dụng rộng rãi trong công nghiệp phần mềm hiện nay. Phần mẫu thiết kế trình bày khá đầy đủ những mẫu thông dụng nhằm cung cấp cho sinh viên các kiến thức cơ bản và kỹ năng áp dụng. Nội dung của tài liệu được chia thành 11 Chương:

Chương 1: Khái niệm cơ bản về kiến trúc phần mềm

Giới thiệu một số khái niệm liên quan đến thiết kế và kiến trúc cũng như tính phức tạp của phần mềm. Qua đó thấy được lý do tại sao kiến trúc phần mềm được quan tâm đặc biệt trong việc phát triển các hệ phần mềm cỡ lớn. Đồng thời tập trung trình bày một số khái niệm cơ bản làm cơ sở cho trình bày các chương tiếp theo.

Chương 2: Thiết kế kiến trúc và các kiểu kiến trúc

Chương này tập trung trình bày một số chủ đề liên quan đến tiến trình thiết kế kiến trúc, các nguyên lý thiết kế và các kiểu kiến trúc.

Chương 3: Mô hình hóa kiến trúc

Chương này nhằm trình bày một số khái niệm cơ bản sẽ được sử dụng để thể hiện kiến trúc từ những thành phần đơn giản đến các tính chất phức tạp hơn của hệ thống như các mô hình hành vi. Mô hình kiến trúc có thể biểu diễn bởi ngôn ngữ tự nhiên hay ngôn ngữ rất hình thức và hạn chế ngữ nghĩa như ngôn ngữ mô tả kiến trúc Rapide. Biểu diễn kiến trúc với ngôn ngữ mô hình hóa quen thuộc UML được trình bày khá đầy đủ.

Chương 4: Kiến trúc phần mềm dựa trên thành phần

Nội dung chương này tập trung trình bày mô hình thành phần và đặc trưng của mô hình này để làm cơ sở cho hai chương tiếp theo. Mô hình thành phần với UML được tiếp tục trình bày với biểu đồ thành phần và biểu đồ triển khai.

Chương 5: Mô hình thành phần với EJB

Giới thiệu khung J2EE và EJB, các khái niệm về thành phần EJB và môi trường runtime của nó. Nội dung cũng trình bày về các kiểu của EJB, kết nối và việc triển khai của chúng, giới thiệu các tính năng của các phiên bản EJB và phân biệt giữa lời gọi hàm đồng bộ và không đồng bộ. Nội dung cũng đề cập hướng dẫn từng bước để xây dựng, triển khai và sử dụng

thành phần EJB.

Chương 6: Mô hình thành phần .NET

Giới thiệu khung .NET, các khái niệm chung của các thành phần .NET, các kiểu thành phần .NET, kết nối giữa các thành phần, và cách triển khai chúng, các thành phần cục bộ và phân tán, phân biệt các thành phần kết nối và hợp, gọi phương thức đồng bộ và không đồng bộ. Nội dung cũng trình bày hướng dẫn từng bước để xây dựng, triển khai, và sử dụng các thành phần .NET.

Chương 7: Kiến trúc và mẫu thiết kế

Chương này trình bày tổng quan về khái niệm mẫu thiết kế, các nhóm mẫu thiết kế và những đặc trưng của mẫu thiết kế. Một số vấn đề liên quan đến phân loại, tích hợp cũng đã được đề cập đến. Chi tiết mô hình và cài đặt các mẫu thiết kế dành cho các chương tiếp theo.

Chương 8: Các mẫu thiết kế tạo dựng

Chương này tập trung trình bày nhưng mẫu thiết kế tạo dựng được đề cập đến trong chương 7. Trong mỗi mẫu thiết kế đều có bàn đến lý do cho sự ra đời của mẫu, biểu đồ lớp cho mẫu và tình huống áp dụng cùng cài đặt.

Chương 9: Các mẫu thiết kế cấu trúc

Chương này tập trung trình bày nhưng mẫu thiết kế cấu trúc liên quan đến thiết kế cấu trúc các lớp. Trong mỗi mẫu thiết kế đều có bàn đến lý do cho sự ra đời của mẫu, biểu đồ lớp cho mẫu và tình huống áp dụng cùng cài đặt.

Chương 10: Các mẫu thiết kế hành vi

Chương này tập trung trình bày nhưng mẫu thiết kế hành vi liên quan đến thiết kế các thuật toán. Trong mỗi mẫu thiết kế đều có bàn đến lý do cho sự ra đời của mẫu, biểu đồ lớp cho mẫu và tình huống áp dụng cùng cài đặt.

Chương 11: Case study

Chương này trình bày hai Case Study nhằm minh họa áp dụng một số mẫu vào hai hệ khác nhau là Hệ quản lý Cơ sở dữ liệu và Hệ Quản lý bán sách trực tuyến BookStore.

Tài liệu này được ra đời dựa trên sự đóng góp của nhiều người. Trước hết, tác giả xin bày tỏ lòng biết ơn đến tất cả Thầy-Cô trong Khoa CNTT và các Thầy-Cô trong Hội đồng thẩm định đã có nhiều góp ý xây dựng. Đặc biệt cảm ơn các tác giả tài liệu trích dẫn và các tác giả mã nguồn đã được sử dụng trong tài liệu này. Cảm ơn các bạn sinh viên chuyên ngành Công nghệ phần mềm qua nhiều thế hệ của Học viện đã đóng góp cho tài liệu này. Tác giả xem tài liệu này chỉ là sự lắp ghép các mẫu tài liệu có sẵn được sắp xếp theo quan điểm của mình để giúp sinh viên nhanh chóng và dễ dàng tiếp cận môn học. Để hiểu một cách sâu sắc và đầy đủ hơn những chủ đề trong tài liệu, bạn đọc nên tham khảo các tài liệu gốc được liệt kê trong tài liệu tham khảo và chúng nó thực sự đáng để đọc hơn.

Tác giả

CHƯƠNG 1: KHÁI NIỆM CƠ BẢN VỀ KIẾN TRÚC PHẦN MỀM

Chương này tập trung trình bày một số nội dung sau đây:

- Độ phức tạp phần mềm
- Khái niệm kiến trúc phần mềm
- Một số khái niệm cơ bản khác

1.1 ĐỘ PHỨC TẠP PHẦN MỀM

Phần mềm được hiểu là bao gồm chương trình cùng với dữ liệu và tài liệu liên quan nhằm tự động thực hiện một số chức năng hoặc giải quyết một vấn đề cụ thể nào đó. Thông thường khi nói đến phần mềm người ta thường nhấn mạnh đến chương trình. Do đó, trong tài liệu này hai khái niệm phần mềm và chương trình được sử dụng cùng nhau một cách linh hoạt. Phần mềm thực thi các chức năng của nó bằng cách gửi các lệnh trực tiếp đến phần cứng hoặc cung cấp dữ liệu để phục vụ các chương trình hay phần mềm khác. Phần mềm là một khái niệm trừu tượng và khác với phần cứng ở chỗ là không thể sờ mó hay đụng chạm vào và cũng không bị mòn hay phai mờ theo thời gian, và nó cần phải có phần cứng mới có thể thực thi được. Để hiểu rõ phần mềm và các cách tiếp cận trong phát triển phần mềm, phần này sẽ dành trình bày một số đặc trưng về độ phức tạp của phần mềm.

1.1.1 Độ phức tạp phần mềm là gì?

Ngay từ những ngày sơ khai, ngành khoa học máy tính đã gặp phải những vấn đề liên quan đến sự phức tạp như chọn được một cấu trúc dữ liệu phù hợp, phát triển những thuật toán hiệu quả, cách phân rã các chức năng... Máy tính, mạng máy tính và các thiết bị thông minh di động với những kiến trúc phần cứng khác nhau ngày nay đã nâng độ phức tạp của phần mềm lên một mức độ mới với những kiểu phần mềm khác nhau và kiến trúc khác nhau. Độ phức tạp phần mềm (software complexity) có thể được xem xét từ những quan điểm khác nhau. Nó liên quan mật thiết đến kiến trúc phần mềm như trong định nghĩa sau đây của Taylor đã được nhiều người chấp nhận [18]:

Độ phức tạp là bản chất vốn có của một hệ phần mềm. Nó tỷ lệ với kích cỡ của hệ thống, số lượng thành phần cấu thành hệ thống, kích cỡ và cấu trúc bên trong của mỗi thành phần, số lượng và tính chất phụ thuộc nhau giữa các thành phần.

Để hiểu một cách trực giác hơn các khái niệm “kích cỡ”, “cấu trúc bên trong”, “tính chất phụ thuộc”, chúng ta hãy lấy ví dụ từ những khái niệm của lập trình hướng đối tượng. Kích cỡ của một phần mềm hướng đối tượng có thể đo theo số dòng mã nguồn, số lớp, số các mô đun hay số gói; cấu trúc bên trong và phụ thuộc được thể hiện ở cấu trúc của các lớp, gói và quan hệ giữa các lớp, gói.

1.1.2 Độ phức tạp và thành phần phần mềm

Trong kỹ thuật phần mềm truyền thống, mỗi kiểu tác vụ thực thi bởi hệ thống được xác định bởi một hay tập các thành phần liên quan nhau. Điều này xuất phát từ những nguyên lý cơ bản

về phát triển phần mềm dựa vào các khái niệm: *trừu tượng hóa*, *môđun hóa*, *tách các chức năng*, *cô lập thay đổi*. Rõ ràng rằng một hệ phần mềm với số lượng thành phần lớn hơn sẽ phức tạp hơn so với hệ có số lượng thành phần ít hơn.

Thoạt nhìn để giảm độ phức tạp phần mềm thì chỉ cần tối thiểu hóa số thành phần trong hệ thống. Nghĩa là để giảm số thành phần chúng ta chỉ cần phải gộp nhiều chức năng vào trong một thành phần. Tuy nhiên, việc giảm số thành phần trong hệ thống không đảm bảo rằng độ phức tạp sẽ thấp hơn vì điều này có thể làm tăng thêm sự phức tạp của mỗi thành phần riêng lẻ. Nói cách khác, một hệ thống được cấu tạo bởi các thành phần lớn hơn sẽ phức tạp hơn là được cấu tạo bởi các thành phần nhỏ hơn. Hơn nữa, nhiều nghiên cứu đã chỉ ra rằng độ phức tạp của hệ phần mềm còn liên quan đến cả tương tác giữa các thành phần cũng như cấu trúc bên trong và hành vi của riêng các thành phần đó [18].

Độ phức tạp phần mềm được xem là vấn đề căn bản cần phải được giải quyết và khắc phục bởi các công cụ và phương pháp tạo ra phần mềm. Nhiều nghiên cứu đã chỉ ra rằng chất lượng của các sản phẩm phần mềm cũng như tiến độ dự án phần mềm phụ thuộc rất nhiều vào độ phức tạp phần mềm. Tuy nhiên, cho đến nay chưa có một giải pháp nào thực sự hiệu quả để loại bỏ được độ phức tạp hoặc nâng cao năng suất cũng như chất lượng phần mềm. Điều mà những người phát triển có thể làm được là giảm thiểu hoặc loại bớt tính phức tạp đi mà thôi.

Môđun hóa được xem là nguyên tắc cơ bản để đạt được sự đơn giản trong thiết kế phần mềm và có thể làm giảm thiểu cũng như quản lý được độ phức tạp. Khi đó, độ phức tạp trong cấu trúc và xử lý có thể được tính toán dựa vào sự *liên kết* của các thành phần trong một hệ thống. Nghĩa là nếu một hệ thống hoặc một tiến trình càng bao gồm nhiều thành phần kết nối nhau theo nhiều kiểu cấu trúc khác nhau thì độ phức tạp của hệ thống và các tiến trình đó càng lớn. Khi đó các kết nối sẽ trở thành phụ thuộc lẫn nhau.

Ví dụ, trong một cấu trúc phần mềm, thành phần B gọi là *phụ thuộc* vào thành phần A, nếu sửa đổi thành phần A thì thành phần B cũng sẽ bị ảnh hưởng theo. Vì vậy, cần phải sửa đổi thành phần B theo hướng đã sửa đổi thành phần A để hoàn thiện sản phẩm và không gây lỗi. Nghĩa là, nếu thay đổi thành phần A thì cũng phải thay đổi thành phần B. Trong trường hợp hai thành phần này phụ thuộc lẫn nhau tức là thành phần A phụ thuộc vào thành phần B và thành phần B cũng phụ thuộc vào thành phần A thì sự thay đổi của một trong hai thành phần này sẽ dẫn đến một vòng lặp sửa đổi. Điều này dễ hiểu vì khi chỉnh sửa A thì sau đó phải chỉnh sửa B và việc chỉnh sửa B lại dẫn đến việc phải chỉnh sửa A và quá trình này cứ lặp đi lặp lại.

1.1.3 Ảnh hưởng của độ phức tạp phần mềm

Độ phức tạp nảy sinh qua nhiều pha phát triển và thể hiện ở nhiều công đoạn trong quá trình phát triển phần mềm, bao gồm:

- Pha xác định và phân tích yêu cầu
- Phát thảo giao diện người dùng
- Thiết kế kiến trúc
- Thiết kế chi tiết
- Tạo mã nguồn

- Tích hợp...

Chúng ta có thể làm giảm độ phức tạp của thiết kế hệ thống bằng cách lựa chọn ngôn ngữ thiết kế hay chọn giao diện phù hợp hoặc cả hai. Nếu giảm thiểu được một câu hay thuật ngữ của một định nghĩa bằng cách thay đổi ngữ nghĩa của những từ cần được mô tả thì rõ ràng có thể giảm được độ phức tạp. Với ngôn ngữ tự nhiên, có thể giảm độ phức tạp bằng cách đưa vào các thuật ngữ chuẩn. Một từ duy nhất hoặc một tên hợp lý có thể chứa đựng rất nhiều thông tin. Như vậy, *toàn bộ công việc nhằm giảm độ phức tạp phần mềm xoay quanh việc làm sao tạo ra các thuật ngữ để thể hiện các khái niệm và các mối liên quan giữa các khái niệm.*

Cho đến nay, giới công nghệ phần mềm đã đề xuất một số khái niệm như *mẫu kiến trúc* (architectural pattern), *mẫu thiết kế* (design pattern), *thành phần* (component) với hy vọng giảm được độ phức tạp này. Tuy nhiên, việc có giảm được độ phức tạp hay không còn phụ thuộc vào điều các mẫu kiến trúc, mẫu thiết kế hay thành phần đã được hiểu một cách rõ ràng và chính xác chưa. Mặc dù cấu trúc bên trong của hệ thống lúc này vẫn còn phức tạp do sự tương tác giữa các thành phần nhưng nó khiến cho hệ thống được nhìn một cách đầy đủ và tổng quát hơn. Cho đến nay *các khái niệm mẫu thiết kế, mẫu kiến trúc hay thành phần thường được sử dụng để mô tả cấu trúc của phần mềm cũng như bàn luận về thiết kế và kiến trúc phần mềm.* Các khái niệm này sẽ được trình bày đầy đủ hơn trong phần còn lại của chương cũng như các chương sau và có thể xem là những chủ đề xuyên suốt trong tài liệu này.

1.2 KIẾN TRÚC PHẦN MỀM

Thuật ngữ *kiến trúc phần mềm* (software architecture) lần đầu tiên được sử dụng¹ vào cuối những năm 1960 nhưng mãi đến những năm 1990 khái niệm này mới được xem xét, nghiên cứu rộng rãi trong công nghệ phần mềm. Kiến trúc phần mềm được hiểu như một khái niệm quan trọng bắt nguồn từ những nghiên cứu của Dijkstra vào năm 1968 và Parnas vào đầu những năm 1970. Các nghiên cứu này đã chỉ ra rằng *kiến trúc của hệ phần mềm đóng một vai trò quan trọng và làm thế nào có được một kiến trúc hợp lý là vấn đề cốt lõi trong quá trình phát triển phần mềm.* Những năm 1990 đã chứng kiến nhiều nỗ lực nghiên cứu nhằm đề xuất các kiểu kiến trúc phần mềm, các ngôn ngữ mô tả kiến trúc, các mẫu thiết kế, các mô hình thành phần phần mềm, các phương pháp hình thức để mô hình hóa kiến trúc... Động lực chính cho sự quan tâm mạnh mẽ này là mã nguồn của nhiều hệ phần mềm phổ biến đã lên đến hàng trăm nghìn hay hàng triệu dòng lệnh.

Kiến trúc hệ thống (system architecture) là một sự gắn kết các bộ phận trong một hệ thống với nhau bao gồm cấu trúc, giao diện và các cơ chế để phối hợp hoạt động giữa các bộ phận. Khi xác định được kiến trúc phù hợp, chúng ta sẽ dễ dàng chuyển đổi, tìm được vị trí của một chức năng hay chỉ ra được nơi có thể thêm một chức năng mới phù hợp với kiến trúc chung. Một kiến trúc tốt [18] phải được chi tiết hóa đủ để có thể ánh xạ thành mã nguồn thực sự và cho phép thêm những chức năng mới hay những khái niệm mới mà không ảnh hưởng đến phần còn lại của hệ thống.

¹ <http://homepages.cs.ncl.ac.uk/brian.randell/NATO/nato1968.PDF>

Kiến trúc một hệ phần mềm được hiểu là một quá trình xác định một giải pháp để đưa ra một cấu trúc thích hợp nhằm đáp ứng được các yêu cầu về kỹ thuật và hoạt động của một tổ chức hay doanh nghiệp đồng thời có thể tối ưu hóa về mặt hiệu năng, an toàn và khả năng quản lý.

Quá trình này liên quan đến một loạt các quyết định và mỗi quyết định có thể có tác động thực sự đến chất lượng, hiệu năng, bảo trì, và thành công chung của một hệ ứng dụng. Để hiểu rõ hơn khái niệm kiến trúc, chúng ta sẽ xem xét một số định nghĩa đã được đưa ra gần đây [13]:

- Booch đã định nghĩa kiến trúc phần mềm là một loạt các quyết định quan trọng về tổ chức một hệ thống phần mềm: Lựa chọn các thành phần của cấu trúc, giao diện của hệ thống; Xác định kiểu tương tác giữa các thành phần; Xác định các yếu tố cơ bản cấu thành hệ thống; Xác định kiểu kiến trúc của hệ thống; Khả năng tái xử dụng...
- Martin Fowler cũng đã chỉ ra một số chủ đề phổ biến khi giải thích về kiến trúc như sau: việc phân rã hệ thống ở mức cao thành các bộ phận, những quyết định khó thay đổi, nhiều kiến trúc trong hệ thống, cái gì có thể thay đổi trong vòng đời của hệ thống...
- Bass et al. [3] đã định nghĩa kiến trúc phần mềm là một hay nhiều cấu trúc của hệ thống bao gồm các thành phần phần mềm và các quan hệ giữa chúng.

Như vậy, kiến trúc một hệ thống là tìm cách xây dựng một liên kết giữa các yêu cầu nghiệp vụ và các yêu cầu kỹ thuật bằng việc hiểu các ca sử dụng (use case) và sau đó tìm cách cài đặt những ca sử dụng này vào trong phần mềm.

Mục tiêu của kiến trúc là chỉ ra được các yêu cầu có ảnh hưởng như thế nào đến cấu trúc của ứng dụng. Kiến trúc tốt có thể giảm được rủi ro về nghiệp vụ trong khi xây dựng một giải pháp kỹ thuật và có thể xử lý được các sự cố xảy ra liên quan đến phần cứng hay phần mềm trong vòng đời của nó. Kiến trúc sư phần mềm cần phải xem xét ảnh hưởng tổng thể của những quyết định thiết kế, sự cân bằng vốn có của các thuộc tính chất lượng (như hiệu năng và bảo mật) và sự cân bằng cần thiết của yêu cầu người dùng, hệ thống và yêu cầu nghiệp vụ. Sau đây là một số hướng dẫn khi kiến trúc một hệ thống:

- Kiến trúc cần thể hiện được cấu trúc (structure) của hệ thống nhưng ẩn giấu các chi tiết cài đặt.
- Kiến trúc phải hiện thực hóa được tất cả các ca sử dụng và kịch bản.
- Kiến trúc phải thể hiện được sự quan tâm đến các yêu cầu của các bên liên quan.
- Kiến trúc phải đáp ứng cả yêu cầu về chức năng lẫn yêu cầu về phi chức năng

1.3 MỘT SỐ KHÁI NIỆM CƠ BẢN

Trong các phần trình bày trên, chúng ta đã bàn một số vấn đề liên quan đến khái niệm của kiến trúc phần mềm như tính phức tạp, thành phần...Phần này nhằm trình bày chi tiết hơn một số định nghĩa về các thuật ngữ và những ý tưởng cơ bản về kiến trúc phần mềm để làm cơ sở cho trình bày các chương về sau.

1.3.1 Kiến trúc

Theo Taylor [18] *kiến trúc một hệ phần mềm là một tập các quyết định thiết kế chủ yếu được đưa ra cho hệ thống cần xây dựng*. Như vậy, kiến trúc phần mềm được xem là những định hướng cho phát triển và tiến hóa một hệ phần mềm. Khái niệm *quyết định thiết kế* (design decision) là trung tâm của kiến trúc phần mềm nhằm kết nối các khái niệm khác liên quan đến kiến trúc. Sau đây là một số khái niệm liên quan đến quyết định thiết kế:

- Các quyết định liên quan đến *cấu trúc* (structure) của hệ thống. Ví dụ, các thành phần kiến trúc cần phải tổ chức và hợp thành các gói phân cấp theo kiểu 3 tầng.
- Các quyết định liên quan đến *hành vi chức năng* (functional behavior). Ví dụ, xử lý dữ liệu, lưu trữ và hiển thị sẽ được thực thi theo một thứ tự nghiêm ngặt.
- Các quyết định liên quan đến *tương tác* (interaction). Ví dụ, giao tiếp giữa các thành phần hệ thống chỉ xảy ra bằng cách sử dụng thông báo, sự kiện hay giao thức RMI trong java.
- Các quyết định liên quan đến *các tính chất phi chức năng* (nonfunctional properties) của hệ thống. Ví dụ, khả năng phụ thuộc của hệ thống sẽ được đảm bảo bằng cách phục hồi các môđun xử lý.
- Các quyết định liên quan đến *cài đặt* (implementation) hệ thống. Ví dụ, thành phần giao diện người sử dụng sẽ được xây dựng bằng cách sử dụng Java Swing hay JSP.

Rõ ràng các quyết định quan trọng khác nhau sẽ dẫn đến các kiến trúc khác nhau. Tuy nhiên, một số quyết định không được xem là chủ yếu và không ảnh hưởng đến kiến trúc của hệ thống như chi tiết của thuật toán hay cấu trúc dữ liệu.

1.3.2 Thành phần

Các quyết định tạo nên kiến trúc một hệ phần mềm bao gồm việc kết hợp nhiều phần tử khác biệt nhưng có liên quan với nhau. Các phần tử này bao gồm những khía cạnh khác nhau:

- Xử lý: liên quan đến chức năng hay hành vi
- Trạng thái: liên quan đến thông tin hay dữ liệu
- Tương tác: liên quan đến cộng tác, phối hợp...

Những phần tử đóng gói xử lý và dữ liệu bên trong kiến trúc hệ thống gọi là *thành phần phần mềm* (software component). Theo Taylor [18]:

Thành phần phần mềm là một thực thể kiến trúc: (i) đóng gói một tập con chức năng và dữ liệu của hệ thống; (ii) chỉ có thể truy nhập vào thực thể thông qua một giao diện; (iii) xác định các phụ thuộc tùy theo ngữ cảnh mà nó yêu cầu.

Như vậy, thành phần phần mềm là sự tích hợp của tính toán và trạng thái trong hệ thống. Tùy theo kiểu kiến trúc, quan điểm người thiết kế hay yêu cầu của hệ thống, thành phần có thể đơn giản như một thao tác, một phương thức hay phức tạp như một phần hệ thống. Đặc trưng quan trọng nhất của thành phần là chỉ được nhìn thấy từ bên ngoài qua giao diện. Thành phần phần mềm được cho là thể hiện đầy đủ các nguyên lý *đóng gói* (encapsulation), *trừu*

tượng hóa (abstraction) và *môđun hóa* (modularity) của kỹ thuật phần mềm. Như vậy, các thành phần hàm ý có khả năng kết hợp, sử dụng lại và tiến hóa.

Tuy nhiên, các khả năng này lại được thể hiện trong ngữ cảnh tạo ra thành phần như giao diện phần mềm, tính có sẵn của tài nguyên (dữ liệu...), thùng chứa thành phần, môi trường thực thi của chương trình, hệ điều hành, giao thức mạng, cấu hình phần cứng...

1.3.3 Kết nối

Một đặc trưng cơ bản quan trọng nữa của hệ phần mềm là *tương tác* (interaction) giữa các thành phần trong hệ thống. Nhiều hệ thống hiện đại được xây dựng với nhiều thành phần phức tạp, phân tán qua mạng và cập nhật liên tục suốt trong vòng đời của nó. Do đó, đối với người phát triển việc đảm bảo tương tác giữa các thành phần thậm chí quan trọng hơn chính chức năng của bản thân các thành phần.

Kết nối là một phần tử kiến trúc có nhiệm vụ thực thi tương tác giữa các thành phần.

Đối với hệ phần mềm trên máy để bàn truyền thống, các kết nối thường được thể hiện bởi lời *gọi thủ tục* hay *truy nhập dữ liệu chung*. Các kết nối này thường chỉ thể hiện sự tương tác giữa các cặp thành phần. Gọi thủ tục được cài đặt trực tiếp trong các ngôn ngữ lập trình để giúp trao đổi đồng bộ dữ liệu và điều khiển giữa các cặp thành phần. Khi đó, thành phần triệu gọi chuyển luồng điều khiển cũng như dữ liệu dưới dạng tham số đến thành phần được gọi và sau khi thực hiện yêu cầu, bên được gọi *trả lại* điều khiển cũng như kết quả tính toán cho bên gọi. Trong truy nhập dữ liệu, kiểu *kết nối* được thể hiện dưới dạng biến toàn cục hay bộ nhớ chung. Kiểu kết nối này cho phép nhiều thành phần tương tác bằng cách đọc hay ghi vào các phương tiện chung. Tương tác được phân bố không đồng bộ theo thời gian nghĩa là việc đọc không phụ thuộc hay ràng buộc theo thời gian của việc ghi và ngược lại.

Tuy nhiên, trong các hệ phần mềm phức tạp trên mạng, các kết nối thường xảy ra cùng một lúc giữa một thành phần với nhiều thành phần dịch vụ khác nhau. Dựa vào cách thực thi dịch vụ tương tác, người ta thường phân các kết nối thành tám kiểu khác nhau [18]:

- *Kết nối gọi thủ tục*: ví dụ như phương thức trong lập trình hướng đối tượng.
- *Kết nối sự kiện*: ví dụ trong ứng dụng windows, các input GUI xem như sự kiện kích hoạt hệ thống.
- *Kết nối truy nhập dữ liệu*: ví dụ các cơ chế truy vấn trong cơ sở dữ liệu SQL.
- *Kết nối liên kết*: được sử dụng để ràng buộc các thành phần cùng giữ một trạng thái suốt trong quá trình tương tác với nhau.
- *Kết nối luồng*: được sử dụng để truyền một khối lượng lớn dữ liệu giữa các tiến trình tự chủ riêng. Ví dụ, các socket TCP/UDP hay các giao thức client-server như RMI, CORBA, FTP, SOAP.
- *Kết nối trung gian*: được sử dụng khi các thành phần biết sự có mặt của các thành phần khác nhưng không thể biết được nhu cầu, trạng thái của nó. Khi đó, bộ phận trung gian sẽ cung cấp dịch vụ phối hợp hay giải quyết tranh chấp...

- *Kết nối thích nghi*: cung cấp phương tiện để hỗ trợ tương tác giữa các thành phần không được thiết kế với tương tác. Ví dụ, hệ phân tán có thể dựa vào gọi thủ tục từ xa (RPC: remote procedure call) cho mọi tương tác.
- *Kết nối theo tuyến*: được sử dụng để chỉ ra các tuyến tương tác và thực thi định tuyến truyền thông cũng như phối hợp giữa các thành phần theo tuyến này.

Thông thường các hệ phân tán sử dụng nhiều cách kết nối với nhau. Ví dụ, trong tính toán lưới, kết nối phân tán dữ liệu dựa trên hợp của bốn kiểu kết nối: gọi thủ tục, truy nhập dữ liệu, luồng và kết nối theo tuyến. Các kết nối này phân phối một khối lượng lớn dữ liệu giữa các thành phần trong môi trường này. Trong các hệ P2P, kết nối phân tán gồm bốn kiểu: trung gian, truy nhập dữ liệu, luồng và kết nối theo tuyến. Trong các hệ phân tán dựa trên mô hình client-server, kết nối gồm bốn kiểu: gọi thủ tục, truy nhập dữ liệu, luồng và kết nối theo tuyến.

1.3.4 Cấu hình

Các thành phần và kết nối thường được kết hợp với nhau theo cách nào đó trong cấu trúc để xác lập mục tiêu của hệ thống. Cách kết hợp này thể hiện cấu hình hay tô pô của hệ thống. Như vậy, *cấu hình của kiến trúc là tập các cách kết hợp giữa các thành phần và kết nối của kiến trúc hệ phần mềm*.

Cấu hình có thể biểu diễn như một đồ thị trong đó các đỉnh biểu diễn thành phần và kết nối, các cạnh biểu diễn sự kết hợp.

1.3.5 Kiểu kiến trúc

Kiểu kiến trúc là tập những quyết định thiết kế: (i) áp dụng được trong ngữ cảnh phát triển; (ii) ràng buộc những quyết định thiết kế cho hệ thống đó; (iii) thể hiện được chất lượng và lợi ích trong kiểu hệ thống như vậy.

Nhiều kinh nghiệm và nghiên cứu đã chỉ ra rằng trong các hệ phân tán, kiểu kiến trúc thỏa một số điều kiện sau đây sẽ đem nhiều lợi điểm:

- Tách biệt về mặt vật lý: các thành phần phần mềm yêu cầu dịch vụ tách biệt với các thành phần cung cấp dịch vụ. Điều này cho phép cấu trúc các thành phần một cách hợp lý, hiệu quả cả về số lượng yêu cầu và cung cấp dịch vụ.
- Trong suốt: làm cho nhà cung cấp không biết định danh của bên yêu cầu để dễ bề cung cấp dịch vụ một cách trong suốt kể cả khi thay đổi bên yêu cầu.
- Cô lập: tách biệt các bên yêu cầu với nhau để cho phép thêm, loại bỏ hay sửa đổi một cách độc lập.

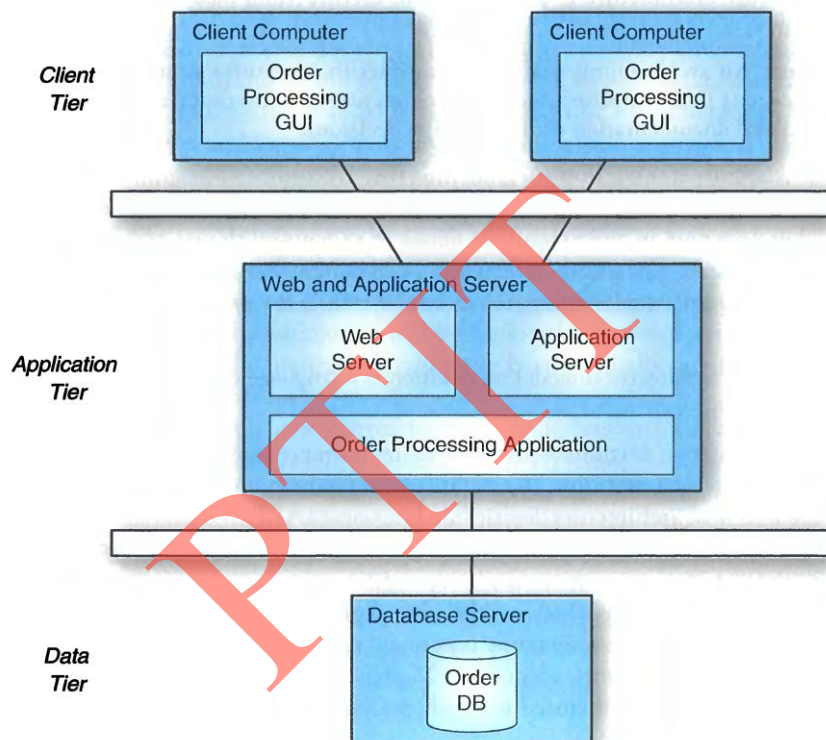
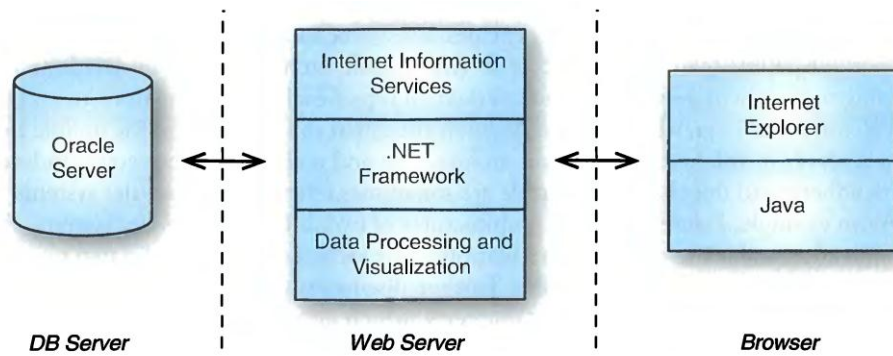
1.3.6 Mẫu kiến trúc

Các kiểu kiến trúc đưa ra những quyết định thiết kế và ràng buộc tổng quát. Chúng cần được mịn hóa thành các quyết định cụ thể dành riêng cho hệ thống cần xây dựng. Trong khi đó, mẫu kiến trúc (architectural pattern) cung cấp một tập các quyết định thiết kế liên quan đến các thành phần và kết nối với những đặc trưng cụ thể cho một lớp hệ phần mềm. Sự phân biệt

giữa mẫu và kiểu kiến trúc không phải bao giờ cũng rõ ràng. Một số đặc trưng sau đây có thể giúp ta hiểu hơn sự khác biệt này:

- *Phạm vi*: Kiểu kiến trúc áp dụng vào một ngữ cảnh phát triển như GUI, trong khi mẫu kiến trúc áp dụng vào một bài toán thiết kế đặc biệt như logic nghiệp vụ của hệ thống phải tách biệt với quản trị dữ liệu.
- *Trừu tượng hóa*: Kiểu giúp ràng buộc các quyết định thiết kế kiến trúc về hệ thống và khá trừu tượng nên đòi hỏi sự thể hiện của con người. Trong khi đó, mẫu là phần kiến trúc đã được tham số hóa và có thể xem như một mẫu thiết kế cụ thể.
- *Quan hệ*: Các mẫu không thể sử dụng ngay được mà phải được tham số hóa cho ngữ cảnh của bài toán cụ thể. Nghĩa là một mẫu cũng có thể áp dụng được cho những hệ thống với nhiều kiểu khác nhau. Ngược lại, một hệ thống được thiết kế theo một kiểu nào đó có thể sử dụng nhiều mẫu.

Ví dụ: Mẫu kiến trúc Hệ phân tán 3-tầng đã được sử dụng rộng rãi hiện nay cho các hệ thống phục vụ nghiên cứu khoa học (chẩn đoán ung thư, thiên văn, địa lý, dự đoán thời tiết..), các hệ thống dịch vụ ngân hàng, thương mại điện tử, đặt chỗ du lịch, chăm sóc y tế...Các hệ thống này có thể xử lý, lưu trữ, truy xuất một khối lượng lớn dữ liệu những người dùng khác nhau (Hình 1.1).



Hình 1.1: Biểu diễn kiến trúc 3 tầng [18]

Các hệ thống này gồm 3 tầng:

- Tầng client hay trình diễn chứa chức năng cần thiết để người dùng truy nhập dịch vụ hệ thống. Tầng này chứa giao diện GUI của hệ thống và có thể lấy dữ liệu và thực thi một số xử lý nhỏ. Tầng client thông thường được triển khai trên các máy với khả năng tính toán và lưu trữ hạn chế như máy để bàn hay các thiết bị cầm tay, di động...
- Tầng ứng dụng (application) hay logic nghiệp vụ hay còn gọi là tầng trung gian chứa chức năng chính của ứng dụng. Tầng này chịu trách nhiệm đáp ứng các yêu cầu dịch vụ hay xử lý yêu cầu từ tầng client và truy nhập, xử lý dữ liệu trong tầng dữ liệu. Tầng này được triển khai trên tập các máy chủ mạnh và lưu trữ lớn.

- Tầng dữ liệu (data) chứa chức năng lưu trữ và truy nhập dữ liệu của ứng dụng. Tầng này chứa cơ sở dữ liệu lớn và có khả năng cung cấp nhiều yêu cầu truy nhập dữ liệu song song.

Tương tác giữa các tầng về nguyên lý là tuân theo khuôn dạng “yêu cầu – đáp ứng” nhưng không quy định chi tiết những dạng tương tác này. Nó có thể cài đặt theo kiểu một yêu cầu - một đáp ứng hay nhiều yêu cầu - một đáp ứng và cũng có thể một yêu cầu – nhiều đáp ứng hay đáp ứng dựa trên cập nhật yêu cầu theo chu kỳ.

1.3.7 Mô hình kiến trúc

Kiến trúc của một hệ phần mềm có thể được thể hiện bởi *mô hình kiến trúc* (architectural model) bằng cách sử dụng những ký hiệu mô hình hóa. Mô hình kiến trúc là sản phẩm thể hiện một số hay tất cả những quyết định thiết kế tạo nên kiến trúc của hệ thống. Tập ký hiệu mô hình kiến trúc được gọi là ngôn ngữ mô tả kiến trúc. Ngôn ngữ này có thể là văn bản, đồ họa, biểu đồ... Ví dụ, kiến trúc hệ 3 tầng có thể thể hiện bởi hai tập ký hiệu mô hình hóa khác nhau (Hình 1.1). Mô hình hóa kiến trúc sẽ được trình bày chi tiết trong Chương 3.

1.4 KẾT LUẬN

Chương này đã trình bày một số khái niệm cơ bản liên quan đến kiến trúc phần mềm. Trước hết tài liệu đã đề cập đến khái niệm độ phức tạp phần mềm. Nó là cơ sở cho lý do tại sao kiến trúc phần mềm trở thành chủ đề quan trọng trong phát triển phần mềm. Khái niệm kiến trúc phần mềm cùng những khái niệm liên quan đến kiến trúc như thành phần, kết nối, mẫu thiết kế, mẫu kiến trúc cũng đã được đề cập một cách khái quát. Các khái niệm như vậy sẽ là cơ sở cho xây dựng kiến trúc phần mềm và sẽ được làm sáng tỏ và chi tiết hơn trong các chương sau.

BÀI TẬP

1. Trong một hệ hướng đối tượng, độ phức tạp phần mềm xác định dựa trên biến, phương thức, đối tượng, gói hay cả hệ thống. Trình bày những đặc trưng và độ đo phần mềm ở các mức độ khác nhau này. Tham khảo <http://worldcomp-proceedings.com/proc/p2015/SER6097.pdf>
2. Một hệ thống thương mại điện tử như eBay, Amazon có cấu trúc gồm các thành phần, kết nối, cấu hình... Hãy khảo sát các hệ trên mạng và chỉ ra các yếu tố của các hệ thống này.
3. Hãy trình bày các đặc trưng của các kiểu kết nối và cách cài đặt các kết nối.
4. Hãy liệt kê các kiểu kết nối (giao thức) có thể có giữa các tầng trong kiến trúc 3 tầng. Hãy trình bày thiết kế và cài đặt một số giao thức đó.
5. Trình bày phân loại các mẫu thiết kế và liệt kê tên của các mẫu thiết kế.
6. Phân biệt các khái niệm mẫu thiết kế, cấu trúc, thiết kế, kiến trúc
7. Chỉ ra một số mẫu thiết kế đã được xây dựng trong các thư viện của Java. Các mẫu này có thể được sử dụng để cài đặt những chức năng nào trong các hệ thống như thương mại điện tử, quản lý đăng ký học tín chỉ, quản lý thư viện?

8. Trình bày các mẫu thiết kế DAO và ví dụ code java tương ứng với lớp Book trong Hệ quản lý sách online. Các phương thức có thể là addBook(), updateBook(), deleteBook().
9. Sử dụng Câu 8 và biểu diễn mẫu kiến trúc MVC với các gói tương ứng.
10. Sử dụng công cụ VP để xây dựng các biểu đồ gói, triển khai của các hệ thống như thương mại điện tử, quản lý đăng ký học tín chỉ, quản lý thư viện...

PTIT

CHƯƠNG 2: THIẾT KẾ KIẾN TRÚC VÀ CÁC KIỂU KIẾN TRÚC

Nội dung của chương này tập trung xem xét một số chủ đề sau đây:

- Tiến trình thiết kế kiến trúc
- Các nguyên lý thiết kế
- Các kiểu kiến trúc

2.1 TIẾN TRÌNH THIẾT KẾ KIẾN TRÚC

Phát triển phần mềm dù theo phương pháp luận nào cũng thường bao gồm các công đoạn xác định yêu cầu, phân tích, thiết kế, cài đặt-tích hợp và kiểm chứng. Thiết kế thường lại được chia thành hai giai đoạn là thiết kế kiến trúc và thiết kế chi tiết. Các phương pháp luận ngày nay như UP (Unified Process) và các phương pháp tiến hóa của nó đều dựa trên *phát triển lặp và tăng dần*. Vì vậy, giống như các công đoạn khác, thiết kế kiến trúc cần được tiến hành ngay từ khi khởi đầu dự án. Các hoạt động thiết kế kiến trúc xem như một quá trình lặp của một dãy các bước và mở rộng thêm khi có thông tin mới được phát hiện trong các pha trước đó. Tiến trình thiết kế kiến trúc thông thường gồm các bước cơ bản sau đây ([1][3]):

1. *Hiểu được vấn đề cần giải quyết*
2. *Xác định các phần tử của thiết kế và quan hệ của chúng*
3. *Đánh giá thiết kế kiến trúc*
4. *Biến đổi thiết kế kiến trúc*

Bước đầu tiên được cho là then chốt vì nếu không hiểu rõ bài toán đặt ra sẽ không thể đưa ra giải pháp hữu hiệu. Bước thứ hai nhằm chỉ ra các phần tử thiết kế và sự phụ thuộc của chúng. Bước thứ ba liên quan đánh giá kiến trúc theo yêu cầu thuộc tính của chất lượng kiến trúc.

Rõ ràng, chức năng của một ứng dụng không thể kiểm chứng được từ quá trình phân rã cấu trúc. Tuy nhiên, nhiều thuộc tính chất lượng khác có thể đánh giá được như xem xét kiểu thiết kế hay cài đặt bản mẫu cho tương tác giữa các thành phần quan trọng. Bước thứ tư liên quan đến áp dụng các thao tác thiết kế để biến đổi thiết kế kiến trúc thành một thiết kế mới với những yêu cầu thuộc tính chất lượng tốt hơn thiết kế trước. Bước này có thể lặp lại nhiều lần. Tiến trình thiết kế kiến trúc được xem là sự mở rộng của tiến trình thiết kế kỹ thuật thông thường.

Mục đích của thiết kế kiến trúc là tập trung vào phân rã hệ thống thành các thành phần và tương tác giữa các thành phần sao cho thỏa mãn các yêu cầu chức năng và phi chức năng được đặt ra. Một hệ phần mềm có thể xem như một phân cấp của các quyết định thiết kế trong đó mỗi mức phân cấp có một tập luật thiết kế để ràng buộc hay kết nối các thành phần ở mức đó.

Ví dụ trong kiến trúc kiểu Client/server, các quy luật thiết kế có thể liên quan đến đặc tả giao diện giữa client và server. Mức phân cấp kế tiếp sẽ mô tả mỗi client hay mỗi server như là một tập các thành phần tương tác. Mỗi nhánh phân cấp có những luật thiết kế riêng mà các

nhánh khác không nhìn thấy được. Như vậy phân rã của client là ẩn dấu đối với server và ngược lại. Đầu ra của tiến trình thiết kế kiến trúc là mô tả cấu trúc.

2.1.1 Hiểu được vấn đề cần giải quyết

Nhiều dự án hay sản phẩm phần mềm được cho là không thành công vì nó không giải quyết được vấn đề thực sự trong nghiệp vụ khách hàng hoặc không thỏa mãn lợi tức đầu tư mà phần mềm đem lại. Câu hỏi đặt ra là tại sao phải đầu tư một giải pháp mới mà chẳng đem lại lợi ích gì so với giải pháp cũ. Đa số kỹ sư phần mềm đều tập trung vào các giải pháp kỹ thuật hơn là giải quyết vấn đề cơ bản mà nhà đầu tư yêu cầu mà thường được gọi là *bẫy cài đặt*. Để tránh cái bẫy như vậy, chúng ta nên tự đặt vấn đề lặp đi lặp lại là các quyết định thiết kế đưa ra nhằm giải quyết những vấn đề gì.

2.1.2 Xác định các phần tử thiết kế và quan hệ của chúng

Trong bước này, chúng ta xác lập một phân rã ban đầu của hệ thống dựa trên các yêu cầu chức năng bao gồm các phần tử thiết kế và các phụ thuộc giữa chúng. Kiến trúc của một hệ phần mềm thường được biểu diễn như một tập các module hay hệ thống con mà ta gọi là thành phần.

Theo cách phân cấp, các chương trình trong C hay Java cũng có thể xem như cấu trúc gồm những thành phần mức thấp hơn. Các ngôn ngữ lập trình xác định một tập các thành phần theo các từ khóa và cú pháp của ngôn ngữ đó. Khai báo số nguyên và gán giá trị cho nó cũng có thể xem như thành phần gồm một tập các thành phần mức thấp hơn là những lệnh và mẫu dữ liệu. Điều này tương tự như chip trong máy tính gồm những thành phần là cổng và cổng lại gồm những thành phần mức thấp hơn là các mạch. Như vậy, kiến trúc có thể xem như cấu trúc phân cấp của phần mềm.

Việc chỉ ra các phần tử thiết kế và quan hệ của chúng có thể phân chia hơn nữa thành các bước sau đây:

- Xác định ngữ cảnh hệ thống.
- Xác định các module
- Mô tả các thành phần và kết nối

Hai bước đầu sử dụng kỹ thuật thiết kế trừu tượng cho phép chúng ta tập trung vào mức chi tiết cụ thể mà không phải tập trung vào mọi khía cạnh của hệ thống. Bước thứ ba liên quan đến việc mô tả các thể hiện của các kiểu module trong cấu hình thời gian chạy.

2.1.3 Đánh giá thiết kế kiến trúc

Thiết kế kiến trúc có thể được đánh giá theo những cách khác nhau. Để đánh giá một thiết kế, chúng ta phải hình dung rõ ràng những yêu cầu thuộc tính chất lượng ảnh hưởng bởi thiết kế. Những phán đoán kiểu như “thiết kế phải chạy nhanh” hay “hệ thống phải dễ dàng sửa đổi hay hiệu chỉnh cho nhu cầu mới” chỉ là những yêu cầu ban đầu mà không đủ để đánh giá thiết kế. Đánh giá kiến trúc không phải là đánh giá chính sản phẩm phần mềm mà là đánh giá mô tả về mặt thiết kế kiến trúc của hệ thống. Đánh giá kiến trúc là nhằm đo chất lượng thiết kế sau

khi nó được tạo ra. Không phải bao giờ cũng có thể đánh giá thiết kế để hiểu thuộc tính chất lượng như khả năng chỉnh sửa và hiệu năng.

Thông thường để tạo một hệ thống mà sau này có thể chỉnh sửa được là phân rã hệ thống thành các tầng. Tuy nhiên, các tầng có thể dẫn đến khó hiểu về mặt tính toán. Cho đến nay việc đánh giá kiến trúc phần mềm là dựa vào kiểu không hình thức trong đó các bên liên quan dự án gặp nhau và đưa ra ý kiến của mình.

2.1.4 Biến đổi thiết kế

Bước này được tiến hành sau đánh giá thiết kế. Nếu thiết kế kiến trúc không thỏa mãn đầy đủ yêu cầu thuộc tính chất lượng, thì nó phải được thay đổi cho đến khi thỏa mãn yêu cầu. Như vậy, bắt đầu từ thiết kế hiện thời chúng ta thực hiện phép biến đổi bằng cách sử dụng các toán tử thiết kế, kiểu và mẫu thiết kế. Các toán tử thiết kế cơ bản bao gồm: phân rã hệ thống thành các thành phần; nén một số thành phần thành một thực thể để cải tiến hiệu năng; trừu tượng hóa thành phần để cải tiến khả năng chỉnh sửa và thích nghi; chia sẻ một thành phần với các thành phần khác để cải tiến khả năng tích hợp, chỉnh sửa.

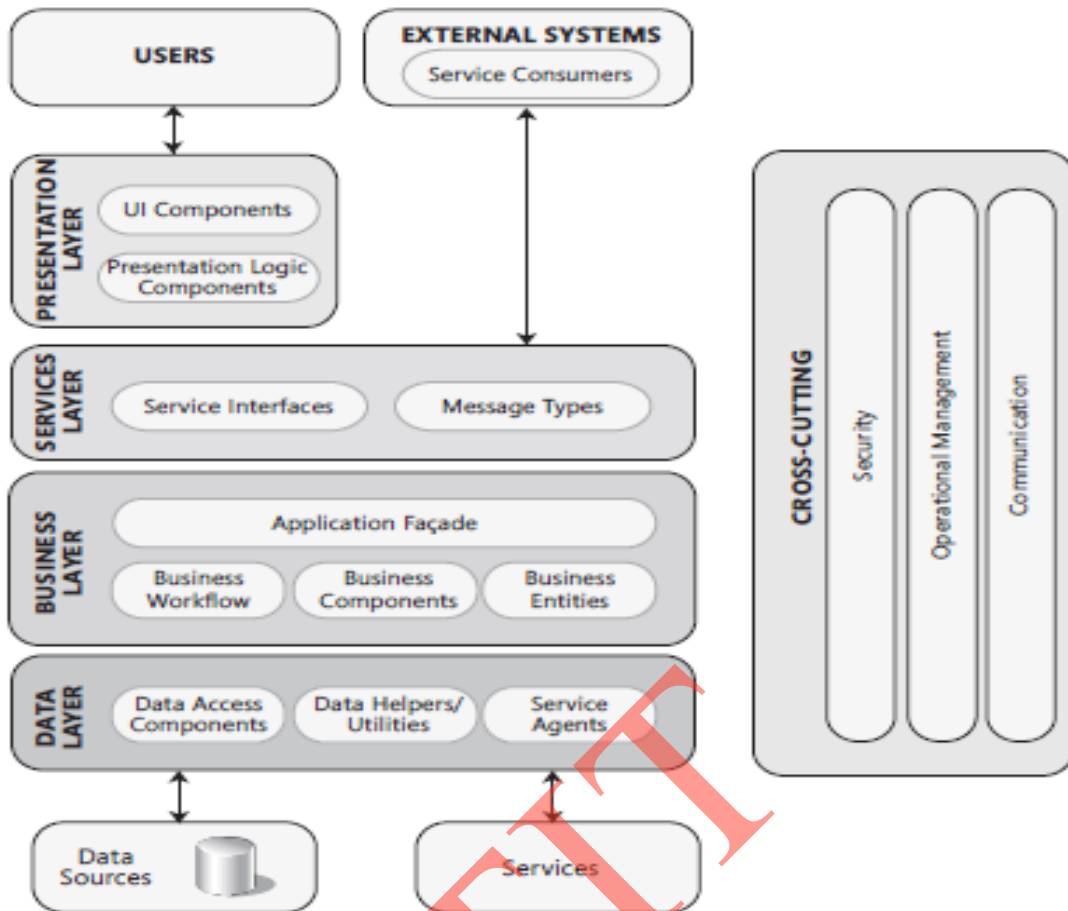
2.2 CÁC NGUYÊN LÝ THIẾT KẾ KIẾN TRÚC

Những nghiên cứu về kiến trúc cho rằng thiết kế phải tiến hóa theo thời gian vì chúng ta không thể biết được mọi điều trong khi tiến hành kiến trúc hệ thống. Do đó, thiết kế cần phải được tiến hóa suốt trong giai đoạn cài đặt ứng dụng vì khi đó chúng ta biết nhiều hơn qua kiểm chứng các yêu cầu thế giới thực. Xây dựng kiến trúc với sự tiến hóa sẽ làm cho nó có thể thích nghi được với yêu cầu chưa được biết đầy đủ trong giai đoạn đầu của quá trình thiết kế.

2.2.1 Kiến trúc phần mềm tổng quát

Một kiến trúc phần mềm thường được mô tả như một cấu trúc hay tổ chức của một hệ thống. Thuật ngữ hệ thống được hiểu như là một tập các thành phần xác lập một chức năng hay một tập các chức năng nào đó. Như vậy, kiến trúc phần mềm tập trung vào tổ chức các thành phần hay nhóm các thành phần thành những cụm có những mục tiêu khác nhau. Kiến trúc tổng quát (Hình 2.1) với các thành phần được nhóm thành những tầng có mục tiêu khác nhau [8]:

- Tầng trình diễn (presentation layer): Bao gồm các thành phần giao diện người dùng (UI: User Interface) và các thành phần logic trình diễn (Presentation logic)
- Tầng dịch vụ (service layer): Bao gồm các giao diện dịch vụ và các kiểu dịch vụ.
- Tầng nghiệp vụ (business layer): Bao gồm các thành phần, thực thể nghiệp vụ và luồng hoạt động nghiệp vụ.
- Tầng dữ liệu (data layer): Bao gồm các thành phần truy nhập dữ liệu, các tiện ích dữ liệu và accs tác nhân dịch vụ.
- Tầng kết nối (cross cutting): Bao gồm an toàn, truyền thông và quản lý điều hành



Hình 2.1: Kiến trúc hệ phần mềm tổng quát [8]

Chương này sẽ xem xét những thao tác thiết kế tổng quát của thiết kế kiến trúc phần mềm và cách sử dụng chúng để phân rã hệ thống thành các thành phần và kết nối để đạt được thuộc tính chất lượng mong muốn. Các thao tác thiết kế thông thường bao gồm: phân rã, gộp, nén, trừu tượng hóa và chia sẻ tài nguyên. Các thao tác này lại được xem xét từ mức độ kiến trúc của thiết kế.

2.2.2 Nguyên lý thiết kế cơ bản

Khi bắt đầu thiết kế, cần phải chú ý đến những nguyên lý cơ bản nhằm giúp tạo ra một kiến trúc có thể tối thiểu hóa được chi phí cũng như yêu cầu bảo trì và tăng cường được khả năng mở rộng. Sau đây là một số nguyên lý cơ bản [8]:

- *Nguyên lý phân rã các chức năng*: Phân chia ứng dụng thành những nhóm sao cho tối thiểu hóa được sự chồng lấp các chức năng. Yếu tố quan trọng là giảm thiểu những điểm tương tác để đạt được *kết hợp cao* (high cohesion) và *kết dính thấp* (low coupling). Tuy nhiên, việc phân rã các chức năng không hợp lý có thể dẫn đến sự kết dính cao giữa các chức năng dù các chức năng liên quan không thực sự chồng lấp.
- *Nguyên lý đơn nhiệm*: Mỗi thành phần hay mô đun nên chịu trách nhiệm cho chỉ một chức năng hay một hợp chức năng có liên kết nhau.

- *Nguyên lý hiểu biết tối thiểu:* Một thành phần hay một đối tượng không nên biết về chi tiết bên trong của các thành phần hay các đối tượng khác.
- *Nguyên lý không tự lập lại:* Chỉ nên đặc tả ý định tại một vị trí. Ví dụ, theo thiết kế ứng dụng, một chức năng cụ thể nên được cài đặt chỉ trong một thành phần. Nghĩa là chức năng đó không nên cài trong những thành phần nào khác.
- *Nguyên lý cực tiểu hóa thiết kế:* Chỉ thiết kế những gì cần thiết. Nguyên lý này đôi khi còn gọi là YAGNI (you ain't gonna need it). Nếu yêu cầu ứng dụng không rõ ràng hay nếu thiết kế cần phải tiến hóa theo thời gian thì tránh đưa ra thiết kế lớn.

Khi thiết kế một hệ thống hay một ứng dụng, kiến trúc sư phần mềm phải cực tiểu hóa độ phức tạp bằng cách tách thiết kế thành những quan tâm khác nhau. Ví dụ, các phần giao diện người dùng, xử lý nghiệp vụ và truy nhập dữ liệu là những thể hiện quan tâm khác nhau. Điều này được thể hiện trong mẫu thiết kế MVC (Model, View, Control) thường gặp.

Trong mỗi lĩnh vực ứng dụng, các thành phần nên tập trung vào một khía cạnh cụ thể và không nên trộn mã nguồn từ những quan tâm khác nhau. Ví dụ, các thành phần xử lý giao diện không nên chứa mã truy nhập trực tiếp nguồn dữ liệu mà nên sử dụng thành phần nghiệp vụ hay thành phần truy nhập dữ liệu để truy xuất dữ liệu.

2.3 CÁC KIỂU KIẾN TRÚC

Phần này dành trình bày những *kiểu kiến trúc* (architectural style) mà chúng được xem như những mẫu và nguyên lý thông dụng trong thiết kế các ứng dụng hiện nay. Với mỗi kiểu kiến trúc, tài liệu sẽ trình bày những nguyên lý chủ yếu, những ưu nhược điểm của từng kiểu kiến trúc nhằm giúp chọn lựa được kiểu kiến trúc phù hợp cho từng ứng dụng.

Việc hiểu biết về các kiểu kiến trúc đem lại nhiều lợi ích trong đó quan trọng nhất là cung cấp một ngôn ngữ chung cho các lớp hệ thống khác nhau. Mỗi kiểu mô tả các khía cạnh khác nhau trong ứng dụng nên thông thường mỗi ứng dụng cụ thể sử dụng một tổ hợp nhiều kiểu khác nhau. Bảng dưới đây liệt kê các khía cạnh chính của các kiểu kiến trúc tương ứng.

Phân loại	Kiểu kiến trúc
Giao tiếp	Kiến trúc hướng dịch vụ, Kiến trúc bus thông điệp
Triển khai	Client/Server, N-tầng, 3-Tầng
Miền	Thiết kế hướng miền
Cấu trúc	Dựa vào thành phần, Hướng đối tượng, Kiến trúc theo tầng

2.3.1 Kiểu kiến trúc Client/Server

Kiến trúc client/server mô tả hệ thống phân tán bao gồm các hệ thống máy chủ, các máy khách và các mạng lưới kết nối. Dạng đơn giản nhất của hệ thống client/server bao gồm một ứng dụng máy chủ được truy cập trực tiếp bởi nhiều máy khách như kiểu kiến trúc 2 tầng (2-tiers). Trước đây, kiến trúc client/server được biểu thị trong một ứng dụng có giao diện đồ họa được giao tiếp với server dữ liệu bao gồm rất nhiều logic nghiệp vụ dưới dạng thủ tục hoặc với một file trên server. Tuy nhiên, kiểu kiến trúc này lại mô tả mối quan hệ giữa một client với một hoặc nhiều server, trong đó client khởi tạo nhiều yêu cầu và gửi đến server sau đó đợi

trả lời từ server và xử lý thông tin nhận được. Server có thể gửi câu trả lời bằng cách sử dụng một loạt các giao thức và định dạng dữ liệu để truyền thông tin cho client.

Ngày nay, nhiều ứng dụng kiểu kiến trúc client/server bao gồm: Các chương trình dựa trên trình duyệt Web chạy trên Internet hoặc một mạng nội bộ như Microsoft Windows - ứng dụng hệ điều hành dựa trên truy cập các dịch vụ dữ liệu trên mạng; ứng dụng truy cập các ngân hàng dữ liệu từ xa (ví dụ như đọc e-mail, FTP khách hàng và các công cụ truy vấn cơ sở dữ liệu); các công cụ và tiện ích thao tác hệ thống từ xa (ví dụ như các công cụ quản lý hệ thống và các công cụ giám sát mạng). Các biến thể từ kiểu kiến trúc client/server bao gồm:

- Các hệ hàng đợi-client: Cách tiếp cận này cho phép client giao tiếp với client khác thông qua một hàng đợi dựa trên server. Client có thể đọc dữ liệu từ server và gửi dữ liệu đến một server và sử dụng một hàng đợi để lưu trữ dữ liệu. Điều này cho phép client phân phối và đồng bộ hóa các tập tin và thông tin. Điều này đôi khi được gọi là một kiến trúc hàng đợi thụ động.
- Hệ Peer-to-Peer (P2P) được phát triển từ kiểu hệ hàng đợi - client. Kiểu P2P cho phép client và server trao đổi vai trò của chúng trong quá trình phân phối và đồng bộ các file và thông tin thông qua nhiều client. Kiến trúc này mở rộng kiểu client/server thông qua việc có nhiều phản hồi cho các yêu cầu, chia sẻ dữ liệu, khai thác tài nguyên, và khả năng phục hồi để loại bỏ các bên.
- Ứng dụng server: là một kiểu kiến trúc đặc biệt trong đó server lưu trữ và thực hiện các ứng dụng và dịch vụ mà một client truy cập thông qua một trình duyệt hoặc một client được cài đặt phần mềm đặc biệt. Ví dụ như một client thực hiện một ứng dụng chạy trên server thông qua nền tảng như dịch vụ đầu cuối.

Những ưu/nhược điểm của kiểu kiến trúc client/server

- Có tính bảo mật cao: Tất cả dữ liệu được lưu trên server, nơi thường có sự quản lý, an toàn cao hơn các máy client.
- Truy cập dữ liệu tập trung: Bởi vì dữ liệu chỉ được lưu trữ trên server nên việc truy cập và cập nhật dữ liệu sẽ nằm dưới quyền người quản trị.
- Dễ dàng bảo trì: Vai trò và trách nhiệm của một hệ thống máy tính được phân bố trong một số server được biết đến với nhau thông qua một mạng lưới. Điều này đảm bảo cho các máy client không bị ảnh hưởng khi một máy server bị sửa chữa, nâng cấp hoặc di chuyển.
- Tuy nhiên, kiểu kiến trúc này cũng có một số bất tiện như việc xây dựng ứng dụng có dữ liệu và logic nghiệp vụ được kết hợp chặt chẽ trên các server. Điều này có thể tác động tiêu cực đến việc mở rộng hệ thống và dẫn đến sự phụ thuộc vào một server trung tâm, ảnh hưởng đến độ tin cậy của hệ thống. Để giải quyết những vấn đề này, kiểu kiến trúc client-server đã phát triển thành kiểu kiến trúc 3-tầng (hoặc N-tầng) sau này.

Hướng dẫn sử dụng

Trong quá trình thiết kế chúng ta cần cân nhắc sử dụng kiểu kiến trúc client/server nếu ứng dụng đang xây dựng dựa trên máy chủ và hỗ trợ nhiều khách hàng. Ví dụ, khi tạo ra các ứng

dụng web dựa trên một trình duyệt Web, việc xử lý quy trình nghiệp vụ sẽ được sử dụng bởi những người trong cùng một tổ chức, hoặc tạo ra các dịch vụ cho các ứng dụng khác sử dụng. Kiểu kiến trúc client/server cũng thích hợp cho rất nhiều kiểu mạng nếu muốn tập trung lưu trữ hoặc sao lưu dữ liệu và các chức năng quản lý, hoặc khi ứng dụng cần phải hỗ trợ các loại client và thiết bị khác nhau.

2.3.2 Kiểu kiến trúc dựa vào thành phần

Kiểu kiến trúc dựa vào thành phần thể hiện phương pháp tiếp cận thành phần cho thiết kế hệ thống. Nó tập trung vào phân rã kiến trúc thành các thành phần chức năng riêng biệt hoặc các thành phần tiếp xúc với giao diện chuẩn qua kết nối. Cách tiếp cận như vậy đem lại một mức độ trừu tượng cao hơn so với phương pháp thiết kế hướng đối tượng. Nguyên tắc chính của kiểu kiến trúc dựa vào thành phần là việc sử dụng các thành phần với các tính chất:

- *Khả năng tái sử dụng*: các thành phần thường được thiết kế để tái sử dụng trong những hoàn cảnh khác nhau với những ứng dụng khác nhau. Tuy nhiên, cũng có một số thành phần có thể chỉ được thiết kế để làm một công việc cụ thể.
- *Khả năng thay thế được*: Các thành phần có thể được thay thế bằng các thành phần tương tự.
- *Không có hoàn cảnh cụ thể*: Các thành phần hoạt động dưới những hoàn cảnh và môi trường khác nhau. Các thông tin cụ thể như dữ liệu trạng thái nên được chuyển về thành phần thay vì được bao gồm hoặc truy cập bởi thành phần này.
- *Khả năng mở rộng*: một thành phần có thể được mở rộng từ một thành phần cũ để cung cấp thêm các trạng thái và hoạt động mới.
- *Được đóng gói*: Thành phần tiếp xúc với giao diện cho phép người gọi sử dụng chức năng của nó và không tiết lộ chi tiết về tiến trình nội bộ hoặc bất kỳ một biến hoặc trạng thái nội bộ.
- *Tính độc lập*: Các thành phần được thiết kế để chỉ phụ thuộc tối thiểu vào các thành phần khác. Do đó, các thành phần có thể được triển khai trong bất kỳ môi trường thích hợp mà không ảnh hưởng đến các thành phần hoặc các hệ thống khác.

Các loại thành phần phổ biến được sử dụng trong các ứng dụng bao gồm các thành phần giao diện người dùng như hệ thống điều khiển với nút bấm; các thành phần trợ giúp hay thành phần tiện ích tiếp xúc với một nhóm cụ thể các chức năng được sử dụng trong các thành phần khác. Các loại thành phần phổ biến khác là những thành phần có nguồn tài nguyên riêng biệt, không được truy cập thường xuyên; các thành phần hàng đợi mà các phương thức có thể được thực hiện không đồng bộ bằng cách sử dụng thông điệp xếp hàng để lưu trữ hoặc chuyển tiếp.

Các thành phần phụ thuộc vào một cơ chế trên nền tảng cung cấp môi trường thực hiện cho chúng thường được gọi là kiến trúc thành phần. Ví dụ như mô hình đối tượng thành phần COM và thành phần phân tán DCOM trong .NET; CORBA hay EJB trong J2EE trên các nền tảng khác. Kiến trúc thành phần quản lý các cơ chế định vị các thành phần và giao diện của chúng, thông qua các thông điệp hoặc lệnh giữa các thành phần, và trong một số trường hợp duy trì trạng thái.

Tuy nhiên, các thuật ngữ trong thành phần thường được sử dụng theo nghĩa cơ bản nhiều hơn là một phần cấu thành, phần tử, hoặc thành phần. Khung làm việc .NET của Microsoft, Khung làm việc J2EE của Java hỗ trợ xây dựng ứng dụng bằng cách sử dụng cách tiếp cận dựa trên thành phần.

Những ưu điểm của kiểu kiến trúc dựa trên thành phần

Sau đây là những lợi ích chính của kiểu kiến trúc dựa trên thành phần:

- *Dễ triển khai:* Các phiên bản tương thích có sẵn nên có thể thay thế phiên bản hiện tại mà không ảnh hưởng đến các thành phần khác hoặc hệ thống nói chung.
- *Chi phí giảm:* Việc sử dụng các thành phần của bên thứ ba cho phép giảm chi phí phát triển và bảo trì.
- *Dễ dàng phát triển:* Các thành phần thực hiện các giao diện nổi tiếng cung cấp chức năng đã được xác định nên cho phép phát triển mà không ảnh hưởng các bộ phận khác của hệ thống.
- *Tái sử dụng:* Việc sử dụng các thành phần tái sử dụng có nghĩa là chúng có thể được sử dụng để giảm chi phí phát triển và chi phí bảo trì qua nhiều ứng dụng hoặc hệ thống.
- *Giảm thiểu sự phức tạp kỹ thuật:* Thành phần giảm thiểu sự phức tạp thông qua việc sử dụng vật chứa thành phần và dịch vụ của nó. Dịch vụ thành phần bao gồm kích hoạt thành phần, phương thức xếp hàng, các sự kiện, và các giao dịch.

Hướng dẫn sử dụng

Trong thiết kế, ta chọn kiểu kiến trúc dựa trên thành phần nếu có các thành phần phù hợp hoặc có thể có được các thành phần phù hợp từ nhà cung cấp bên thứ ba. Khi đó ứng dụng sẽ chủ yếu thực hiện chức năng thủ tục và với dữ liệu ít hoặc không có đầu vào; hoặc khi muốn có thể kết hợp các thành phần viết bằng ngôn ngữ mã khác nhau. Ta cũng có thể sử dụng kiểu này khi muốn tạo ra một kiến trúc hỗn hợp, cho phép thay thế và cập nhật dễ dàng các thành phần riêng rẽ.

2.3.3 Kiểu kiến trúc hướng miền

Thiết kế hướng miền là một cách tiếp cận hướng đối tượng để thiết kế phần mềm dựa trên các hoạt động nghiệp vụ với các thuộc tính, hành vi và quan hệ giữa chúng. Nó cho phép các hệ thống phần mềm thực hiện những chức năng cơ bản bằng cách xác định một mô hình miền thể hiện trong ngôn ngữ của các chuyên gia trong lĩnh vực nghiệp vụ. Mô hình miền có thể được xem như là một khuôn khổ để hợp lý hóa các giải pháp có thể có được.

Để áp dụng thiết kế dựa trên mô hình miền, chúng ta phải có một hiểu biết khá rõ về lĩnh vực nghiệp vụ mà mình muốn mô hình, hoặc có kỹ năng cũng như kiến thức nghiệp vụ liên quan. Đội ngũ phát triển sẽ thường xuyên làm việc với các chuyên gia trong lĩnh vực nghiệp vụ để mô hình miền cho đúng. Kiến trúc sư, nhà phát triển và chuyên gia thường xuất phát từ những môi trường khác nhau nên ngôn ngữ cũng như yêu cầu của họ khi mô tả các mục tiêu, thiết kế thường không giống nhau. Tuy nhiên, trong thiết kế hướng miền, các nhóm cần thống

nhất chỉ sử dụng một ngôn ngữ tập trung vào lĩnh vực nghiệp vụ hơn là các thuật ngữ kỹ thuật.

Cốt lõi của bất kỳ phần mềm nào cũng là mô hình miền. Việc tạo ra một ngôn ngữ chung không chỉ là để thể hiện thông tin từ các chuyên gia lĩnh vực đó mà còn là để áp dụng nó. Những vấn đề truyền thông trong nhóm phát triển không phải do hiểu lầm về ngôn ngữ của miền, mà thực sự là do ngôn ngữ để mô hình bản thân miền đó không rõ ràng. Các tiến trình thiết kế hướng miền nhắm đến mục tiêu không chỉ đưa các ngôn ngữ mà còn cải thiện và tinh chỉnh ngôn ngữ của miền. Điều này sẽ mang lại lợi ích cho phần mềm được xây dựng, vì mô hình là phép chiếu trực tiếp của ngôn ngữ miền.

Để giúp duy trì các mô hình như là một cấu trúc ngôn ngữ, chúng ta thường phải thực hiện rất nhiều cách phân rã và đóng gói trong mô hình miền. Do đó, một hệ thống dựa trên mô hình hướng miền sẽ khiến cho chi phí tăng cao. Việc thiết kế hướng miền đem lại nhiều lợi ích kỹ thuật như bảo trì, nhưng thường chỉ được áp dụng với các lĩnh vực phức tạp đòi hỏi có sự hiểu biết chung về miền.

Những ưu điểm của kiểu kiến trúc hướng miền

- *Truyền thông liên lạc:* Tất cả các thành viên trong các nhóm phát triển có thể sử dụng mô hình miền và các đối tượng được định nghĩa để truyền đạt kiến thức nghiệp vụ với một ngôn ngữ nghiệp vụ thông thường mà không cần các thuật ngữ kỹ thuật.
- *Mở rộng:* Mô hình miền thường là các mô-đun và linh hoạt, nó dễ dàng cập nhật và mở rộng khi các điều kiện và yêu cầu thay đổi.
- *Kiểm chứng:* Các đối tượng mô hình miền gắn kết nhau một cách lỏng lẻo nên chúng dễ dàng được kiểm định.

Hướng dẫn sử dụng

Nên sử dụng kiến trúc hướng miền nếu chúng ta phải xây dựng phần mềm cho một lĩnh vực phức tạp cũng như muốn cải thiện truyền thông và sự hiểu biết trong đội ngũ phát triển. Hoặc trong trường hợp chúng ta phải thể hiện các thiết kế của một ứng dụng trong một ngôn ngữ phổ biến mà tất cả các bên liên quan có thể hiểu được. Kiến trúc kiểu này cũng có thể là một phương pháp lý tưởng nếu bạn có kịch bản dữ liệu nghiệp vụ lớn và phức tạp mà khó có thể quản lý bằng cách sử dụng các kỹ thuật khác.

2.3.4 Kiểu kiến trúc phân tầng

Kiến trúc phân tầng nhằm các nhóm chức năng có liên quan trong một ứng dụng thành các tầng riêng biệt được xếp chồng lên nhau theo chiều dọc. Chức năng trong các tầng kết nối với nhau bởi vai trò hay trách nhiệm chung và khi đó thông tin liên lạc giữa các tầng là rõ ràng và liên kết lỏng lẻo. Việc phân tầng nếu áp dụng một cách đúng đắn sẽ giúp cho việc phân rã hệ phức tạp tốt hơn và sẽ hỗ trợ tính linh hoạt và bảo trì sau này. Kiểu kiến trúc phân tầng được mô tả như là một kim tự tháp ngược của việc tái sử dụng trong đó mỗi tầng tập hợp các nhiệm vụ và trừu tượng hóa tầng trực tiếp bên dưới nó. Với việc phân tầng nghiêm ngặt, các thành phần trong một tầng chỉ có thể tương tác với các thành phần trong chính tầng đó hoặc với các thành phần từ các tầng trực tiếp bên dưới nó.